

UNIVERSITÉ NUMÉRIQUE D'ABIDJAN (UNA)
ÉCOLE SUPÉRIEURE DES TECHNOLOGIES DE L'INFORMATION
MASTER 2 — MÉTHODES INFORMATIQUES APPLIQUÉES À LA GESTION DES
ENTREPRISES (MIAGE)

MÉMOIRE DE FIN D'ÉTUDES

En vue de l'obtention du diplôme de Master 2 MIAGE

**JOBFINDER AI V2.1 :
CONCEPTION ET DÉVELOPPEMENT D'UNE
PLATEFORME
INTELLIGENTE DE MISE EN RELATION EMPLOI
POUR LA CÔTE D'IVOIRE**

Intégration de l'intelligence artificielle, du scraping automatisé et d'un moteur de
recommandation pour le marché de l'emploi en Afrique de l'Ouest

**Présenté par :
ADOPO Arnold Freddy**

Sous la direction de :
Dr. [Nom de l'encadrant] — Enseignant-chercheur, UNA

Année académique 2024 – 2025

REMERCIEMENTS

Ce mémoire est le fruit d'un travail de plusieurs mois qui n'aurait pu voir le jour sans le soutien précieux de nombreuses personnes. Je tiens à exprimer ma profonde gratitude à toutes celles et ceux qui ont contribué, de près ou de loin, à sa réalisation.

Je remercie en premier lieu mon directeur de mémoire, dont les orientations rigoureuses, les remarques constructives et la disponibilité permanente ont guidé chaque étape de ce travail. Sa vision de la recherche appliquée m'a permis de donner une dimension scientifique solide à ce projet technologique.

Je témoigne toute ma reconnaissance aux équipes pédagogiques du Master MIAGE de l'UNA pour la qualité de l'enseignement dispensé tout au long de ce cursus, et particulièrement aux enseignants des modules de génie logiciel, de bases de données et d'intelligence artificielle qui ont directement nourri les choix techniques de ce projet.

Mes sincères remerciements vont également à la communauté open-source — Django Software Foundation, Groq Inc., et les contributeurs de l'écosystème Python — dont les outils libres ont rendu possible le développement de cette plateforme.

Enfin, je remercie ma famille et mes proches pour leur patience, leur encouragement indéfectible et leur soutien moral tout au long de cette aventure académique. Ce travail leur est dédié.

DÉDICACES

À mes parents,

pour chaque sacrifice consenti en silence.

À tous les jeunes chercheurs d'emploi en Côte d'Ivoire et en Afrique de l'Ouest,
que cette plateforme soit un pas vers un marché de l'emploi plus juste.

À la communauté MIAGE de l'UNA,

pour l'émulation intellectuelle et la fraternité partagée.

RÉSUMÉ

Le marché de l'emploi en Côte d'Ivoire souffre d'une fragmentation structurelle : les offres sont dispersées sur de multiples canaux (LinkedIn, AEJI, sites d'entreprises), les candidats manquent d'outils adaptés à leur contexte local, et les recruteurs peinent à trier efficacement les candidatures. JobFinder AI v2.1 répond à cette problématique en proposant une plateforme web complète, développée avec Django 5 et alimentée par l'intelligence artificielle via l'API Groq (modèle LLaMA 3.3 70B).

La plateforme intègre un moteur de scraping automatisé (APScheduler, BeautifulSoup4, Selenium) qui agrège les offres d'emploi depuis LinkedIn et l'AEJI toutes les six heures, un système de recommandation basé sur un modèle de scoring comportemental (JobInteraction), six outils IA — génération de lettres de motivation, optimisation de CV, préparation aux entretiens, conseil en négociation salariale, coaching de recherche d'emploi et assistant conversationnel — ainsi qu'un espace recruteur permettant la publication et la gestion des offres.

Ce mémoire présente l'ensemble du cycle de vie du projet : analyse du contexte et de la problématique, étude préalable (cahier des charges, spécifications fonctionnelles et non fonctionnelles), conception (modélisation UML, architecture Django MVT, schéma de base de données) et implémentation (technologies, interfaces, tests).

Mots-clés : Django, Intelligence Artificielle, Groq API, LLaMA, Scraping, Recommandation, Emploi, Côte d'Ivoire, MIAGE, Lettre de motivation, CV.

ABSTRACT

The job market in Côte d'Ivoire suffers from structural fragmentation: job offers are scattered across multiple channels (LinkedIn, AEJI, company websites), candidates lack tools adapted to their local context, and recruiters struggle to efficiently sort applications. JobFinder AI v2.1 addresses this challenge by providing a comprehensive web platform built with Django 5 and powered by artificial intelligence through the Groq API (LLaMA 3.3 70B model).

The platform integrates an automated scraping engine (APScheduler, BeautifulSoup4, Selenium) that aggregates job offers from LinkedIn and AEJI every six hours, a recommendation system based on a behavioral scoring model (JobInteraction), six AI tools — cover letter generation, CV optimization, interview preparation, salary negotiation advice, job search coaching, and a conversational assistant — as well as a recruiter space for posting and managing job listings.

This thesis presents the full project lifecycle: context analysis, preliminary study (requirements, functional and non-functional specifications), design (UML modeling, Django MVT architecture, database schema) and implementation (technologies, interfaces, testing).

Keywords: Django, Artificial Intelligence, Groq API, LLaMA, Web Scraping, Recommendation Engine, Employment, Côte d'Ivoire, MIAGE, Cover Letter, CV Optimization.

LISTE DES ABRÉVIATIONS

Abréviation	Signification
AEJI	Agence Emploi Jeunes de Côte d'Ivoire
API	Application Programming Interface
APScheduler	Advanced Python Scheduler
ATS	Applicant Tracking System
BS4	BeautifulSoup4
CRUD	Create, Read, Update, Delete
HTML	HyperText Markup Language
IA	Intelligence Artificielle
JSON	JavaScript Object Notation
LLM	Large Language Model (Grand modèle de langage)
MIAGE	Méthodes Informatiques Appliquées à la Gestion des Entreprises
MVT	Model-View-Template (architecture Django)
MySQL	My Structured Query Language
NLP	Natural Language Processing
ORM	Object-Relational Mapping
PDF	Portable Document Format
REST	Representational State Transfer
SQL	Structured Query Language
TF-IDF	Term Frequency–Inverse Document Frequency
UML	Unified Modeling Language
UNA	Université Numérique d'Abidjan
URL	Uniform Resource Locator

TABLE DES FIGURES

- Figure 1** : Architecture globale de JobFinder AI v2.1
- Figure 2** : Architecture MVT de Django appliquée à JobFinder AI
- Figure 3** : Diagramme de cas d'utilisation — Candidat
- Figure 4** : Diagramme de cas d'utilisation — Recruteur
- Figure 5** : Diagramme de séquence — Génération de lettre de motivation
- Figure 6** : Diagramme de séquence — Scraping et agrégation des offres
- Figure 7** : Diagramme de séquence — Recommandation d'offres (JobInteraction)
- Figure 8** : Diagramme de classes — Modèle de données principal
- Figure 9** : Schéma de la base de données (tables principales)
- Figure 10** : Interface : Page d'accueil et tableau de bord candidat
- Figure 11** : Interface : Outil de génération de lettre de motivation
- Figure 12** : Interface : Optimiseur de CV
- Figure 13** : Interface : Espace recruteur — Publication d'offre
- Figure 14** : Chaîne de fallback Groq API (5 modèles LLM)
- Figure 15** : Modèle de scoring JobInteraction
- Figure 16** : Pipeline de génération de documents PDF/DOCX

LISTE DES TABLEAUX

Tableau 1 : Comparaison des plateformes d'emploi en Côte d'Ivoire

Tableau 2 : Exigences fonctionnelles de JobFinder AI v2.1

Tableau 3 : Exigences non fonctionnelles

Tableau 4 : Modèles LLM disponibles via Groq API (fallback chain)

Tableau 5 : Scoring comportemental — Modèle JobInteraction

Tableau 6 : Applications Django et leurs responsabilités

Tableau 7 : Endpoints REST principaux de l'API interne

Tableau 8 : Technologies et versions utilisées

Tableau 9 : Plan de tests fonctionnels

Tableau 10 : Résultats de tests de performance

SOMMAIRE

Remerciements

Dédicaces

Résumé / Abstract

Liste des Abréviations

Table des Figures — Liste des Tableaux

Introduction Générale

CHAPITRE I : Contexte et Présentation du Projet

I.1 Le marché de l'emploi en Côte d'Ivoire

I.2 Problématique

I.3 Présentation de JobFinder AI v2.1

I.4 Objectifs du mémoire

CHAPITRE II : Étude Préalable

II.1 Analyse de l'existant

II.2 Spécifications fonctionnelles

II.3 Spécifications non fonctionnelles

II.4 Choix technologiques

CHAPITRE III : Conception de la Solution

III.1 Architecture logicielle

III.2 Modélisation UML

III.3 Conception de la base de données

III.4 Conception des modules IA

CHAPITRE IV : Implémentation et Résultats

IV.1 Environnement de développement

IV.2 Implémentation des modules clés

IV.3 Interfaces utilisateur

IV.4 Tests et validation

Conclusion Générale

Bibliographie

Annexes

Table des Matières

INTRODUCTION GÉNÉRALE

L'essor du numérique en Afrique subsaharienne transforme progressivement tous les secteurs économiques, y compris le marché de l'emploi. Selon le rapport de l'Agence Emploi Jeunes de Côte d'Ivoire (AEJI, 2023), plus de 400 000 jeunes entrent chaque année sur le marché du travail ivoirien, dans un contexte où l'accès à l'information sur les opportunités professionnelles reste inégal et morcelé. Les plateformes numériques d'emploi existantes répondent imparfaitement aux spécificités du marché local : langue, secteurs porteurs, réseau géographique et cultures organisationnelles propres à la sous-région.

C'est dans ce contexte que s'inscrit JobFinder AI v2.1 : une plateforme web conçue et développée entièrement pour le marché ivoirien et ouest-africain, intégrant des fonctionnalités d'intelligence artificielle accessibles gratuitement via l'API Groq et les grands modèles de langage (LLM) de la famille LLaMA. Le projet ambitionne de mettre à la portée de chaque candidat des outils autrefois réservés aux grandes entreprises : génération automatique de lettres de motivation optimisées ATS, analyse et amélioration de CV, préparation aux entretiens, et recommandation personnalisée d'offres d'emploi.

Ce mémoire de fin d'études, rédigé dans le cadre du Master 2 MIAGE de l'Université Numérique d'Abidjan, rend compte de l'intégralité du cycle de développement de cette plateforme. Il s'organise en quatre chapitres : le contexte et la problématique (Chapitre I), l'étude préalable avec spécifications et choix technologiques (Chapitre II), la conception architecturale et UML (Chapitre III), et l'implémentation avec validation par les tests (Chapitre IV). Une conclusion générale synthétise les apports, les limites et les perspectives d'évolution.

CHAPITRE I : CONTEXTE ET PRÉSENTATION DU PROJET



I.1 Le marché de l'emploi en Côte d'Ivoire

I.1.1 Portrait macroéconomique

La Côte d'Ivoire est la première économie de l'Union Économique et Monétaire Ouest Africaine (UEMOA), avec un PIB d'environ 70 milliards de dollars en 2023. Malgré une croissance soutenue de 6 à 8 % par an sur la décennie 2010–2020, le taux de chômage des jeunes de 15 à 35 ans demeure préoccupant, oscillant entre 35 et 40 % en milieu urbain selon les données de l'Institut National de la Statistique (INS, 2022). Abidjan concentre à elle seule près de 70 % des offres d'emploi formelles du pays, créant une forte pression démographique et concurrentielle sur le marché du travail.

Le secteur tertiaire (services, télécommunications, banque-assurance, commerce) représente plus de 60 % des emplois du secteur structuré. Le secteur des technologies de l'information et de la communication (TIC) connaît une croissance rapide, portée par l'initiative nationale « Côte d'Ivoire numérique 2025 » et l'installation progressive de câbles sous-marins à haut débit.

I.1.2 Fragmentation des canaux de recrutement

Le recrutement en Côte d'Ivoire se caractérise par une forte dispersion des canaux d'information. Les entreprises publient leurs offres sur LinkedIn (réseau mondial), sur le site de l'AEJI, sur des portails généralistes comme Emploi.ci ou Jobafrica.net, sur leurs sites institutionnels, et souvent par le bouche-à-oreille. Cette fragmentation impose au candidat de multiplier les consultations quotidiennes, sans garantie d'exhaustivité.

Par ailleurs, la qualité des outils mis à disposition des candidats reste limitée : peu de plateformes proposent une aide à la rédaction des documents de candidature adaptée aux standards ivoiriens, et aucune n'intègre de modèle de langage capable de générer des documents personnalisés et résistants à la détection IA.

I.1.3 Rôle de l'AEJI

L'Agence Emploi Jeunes de Côte d'Ivoire (AEJI), créée en 2016 par le gouvernement ivoirien, constitue le principal acteur public de l'intermédiation emploi. Elle gère un portail national d'offres, propose des formations courtes et des programmes d'insertion professionnelle. Cependant, l'absence d'API publique impose le recours au scraping web pour l'agrégation de ses offres, ce que JobFinder AI réalise de manière automatisée.

I.2 Problématique

Comment concevoir et développer une plateforme numérique accessible, intelligente et culturellement adaptée, capable d'agréger automatiquement les offres d'emploi du marché ivoirien, d'assister les candidats dans la préparation de leurs dossiers grâce à l'intelligence artificielle, et d'aider les recruteurs à trouver les profils les plus pertinents, le tout sans coût prohibitif pour les utilisateurs finals ?

Cette problématique se décompose en trois sous-questions :

- Comment agréger en temps quasi-réel des offres d'emploi dispersées sur des sources hétérogènes (LinkedIn, AEJI) sans API officielle ?
- Comment exploiter des modèles de langage (LLM) de pointe pour générer des documents de candidature personnalisés, au style humain et résistants à la détection IA, sans coût d'inférence pour l'utilisateur ?
- Comment construire un moteur de recommandation pertinent à partir de signaux comportementaux implicites, sans nécessiter d'annotation manuelle ?

I.3 Présentation de JobFinder AI v2.1

I.3.1 Vision et périmètre

JobFinder AI v2.1 est une application web Django, développée en Python 3.11, offrant un écosystème complet de recherche d'emploi et de gestion des ressources humaines pour le marché ivoirien. La plateforme s'adresse à trois types d'utilisateurs : les candidats (chercheurs d'emploi), les recruteurs (employeurs, DRH, cabinets de recrutement) et les administrateurs de la plateforme. La version 2.1 intègre un scraper multi-sources, un moteur de recommandation comportemental, six outils IA distincts, et un système de génération de documents professionnels (PDF + DOCX).

I.3.2 Architecture applicative — Vue d'ensemble

L'application est structurée en six modules Django indépendants :

- **core** : Page d'accueil, tableau de bord, analytics, processeurs de contexte globaux
- **accounts** : Authentification, profils candidats et recruteurs, compétences, formations, expériences
- **jobs** : Gestion des offres, moteur de matching NLP (TF-IDF + cosinus), scraper automatisé
- **employers** : Espace recruteur, publication d'offres, gestion des candidatures reçues
- **applications** : Suivi des candidatures, messagerie interne, notifications
- **ai_tools** : Six outils IA, client Groq, génération PDF/DOCX, modèle JobInteraction

I.3.3 Proposition de valeur

La proposition de valeur de JobFinder AI repose sur quatre piliers différenciants :

- **Gratuité totale** : aucun coût pour les candidats, modèle freemium pour les recruteurs

- **IA intégrée** : génération de lettres avec 4 styles, optimisation CV, coaching entretien, via LLaMA 3.3 70B (API Groq gratuite)
- **Agrégation automatique** : scraping quotidien de LinkedIn CI et AEJI, nettoyage et déduplication
- **Adaptation locale** : filtres géographiques couvrant 81 villes et régions ivoiriennes, interface en français

I.4 Objectifs du Mémoire

- **Objectif 1 — Scientifique** : Démontrer la faisabilité de l'intégration de LLM via API dans une application web Django à coût nul d'inférence
- **Objectif 2 — Technique** : Concevoir et implémenter une architecture logicielle modulaire, maintenable et extensible
- **Objectif 3 — Méthodologique** : Appliquer les méthodes MIAGE (UML, cahier des charges, tests) à un projet réel
- **Objectif 4 — Sociétal** : Contribuer à la réduction du fossé numérique dans l'accès aux outils d'aide à la recherche d'emploi en Côte d'Ivoire

CHAPITRE II : ÉTUDE PRÉALABLE



II.1 Analyse de l'Existant

II.1.1 Benchmark des plateformes d'emploi en Côte d'Ivoire

Une analyse comparative des principales solutions disponibles sur le marché ivoirien a été conduite en janvier 2025. Cinq plateformes ont été étudiées selon sept critères : agrégation automatique, outils IA, adaptation locale, génération de documents, espace recruteur, gratuité candidat, et API ouverte.

Tableau 1 : Comparaison des plateformes d'emploi en Côte d'Ivoire

Critère	LinkedIn	AEJI	Emploi.ci	JobFinder AI
Agrégation auto	Non	Non	Non	Oui (scraping)
Outils IA	Limité (premium)	Non	Non	6 outils (gratuits)
Adaptation CI	Partielle	Oui	Oui	Complète
Génération docs	Non	Non	Non	PDF + DOCX
Gratuité candidat	Partielle	Oui	Oui	Totale
API ouverte	Payante	Non	Non	API interne

Cette analyse révèle que JobFinder AI est la seule plateforme à combiner intégralement les sept critères. La principale différenciation réside dans l'intégration d'outils IA gratuits et l'agrégation automatique de sources multiples, deux fonctionnalités absentes des concurrents locaux.

II.1.2 Critique de l'existant

L'analyse de l'existant révèle trois insuffisances majeures :

- **Silos d'information** : chaque plateforme ne référence que ses propres offres. Le candidat doit visiter 4 à 6 sites quotidiennement pour avoir une vision complète du marché.
- **Absence d'assistance IA** : aucune plateforme locale ne propose de génération de lettre de motivation ou d'optimisation de CV basés sur l'IA générative.
- **Inadaptation culturelle des outils IA globaux** : ChatGPT produit des lettres mal adaptées aux conventions françaises formelles attendues en Côte d'Ivoire, et génère des contenus facilement détectés comme IA.

II.2 Spécifications Fonctionnelles

II.2.1 Acteurs et rôles

- **Candidat** : s'inscrit, complète son profil, consulte et postule aux offres, utilise les outils IA, suit ses candidatures
- **Recruteur** : crée un compte entreprise, publie des offres, accède aux candidatures reçues, consulte les profils
- **Administrateur** : gère la plateforme (utilisateurs, offres, catégories), déclenche les scraping manuels

II.2.2 Fonctionnalités principales (Tableau 2)

Tableau 2 : Exigences fonctionnelles de JobFinder AI v2.1

ID	Fonctionnalité	Priorité
F01	Inscription et authentification (candidat / recruteur)	Haute
F02	Gestion de profil candidat (compétences, expériences, formations)	Haute
F03	Consultation et recherche d'offres avec filtres	Haute
F04	Candidature en ligne avec suivi de statut	Haute
F05	Scraping automatique LinkedIn CI + AEJI (toutes les 6h)	Haute
F06	Génération de lettre de motivation (4 styles, anti-IA, ATS)	Haute
F07	Optimisation de CV avec analyse ATS	Haute
F08	Génération de questions d'entretien personnalisées	Moyenne
F09	Conseil en négociation salariale	Moyenne
F10	Coaching de recherche d'emploi	Moyenne
F11	Assistant conversationnel IA	Moyenne
F12	Export de documents en PDF et DOCX	Haute
F13	Recommandation d'offres (moteur JobInteraction)	Haute
F14	Matching NLP candidat-offre (score 0-100)	Haute
F15	Messagerie interne recruteur-candidat	Moyenne
F16	Tableau de bord analytics	Moyenne
F17	Espace recruteur : publication et gestion d'offres	Haute

II.3 Spécifications Non Fonctionnelles

Tableau 3 : Exigences non fonctionnelles

Catégorie	Exigence	Indicateur
Performance	Temps de réponse moyen < 2s pour 95% des requêtes	< 2s (P95)
Performance	Génération de lettre de motivation < 10s	< 10s
Disponibilité	Disponibilité de la plateforme	> 99%
Sécurité	Authentification sécurisée (CSRF, HTTPS)	OWASP Top 10
Scalabilité	Architecture modulaire extensible	6 apps Django
Maintenabilité	Couverture de tests unitaires	> 70%

Accessibilité	Interface responsive (mobile, tablette, desktop)	Bootstrap 5
---------------	--	-------------

II.4 Choix Technologiques

II.4.1 Framework backend — Django 5

Django a été choisi comme framework web principal pour sa maturité (version 5.0, décembre 2023), son ORM intégré, son système d'authentification natif et son administration automatique. L'architecture MVT impose une séparation claire des responsabilités, facilitant la maintenance et les tests. Django est le framework Python le plus utilisé en production pour les applications web d'entreprise.

II.4.2 Intelligence artificielle — Groq API et LLaMA

L'API Groq constitue le cœur du module IA. Groq propose une infrastructure d'inférence LLM ultra-rapide basée sur des puces LPU (Language Processing Unit) propriétaires. Son API est gratuite (avec limites de débit raisonnables) et donne accès aux modèles LLaMA 3.3 70B (modèle principal, haute qualité) et LLaMA 3.1 8B (repli rapide).

Tableau 4 : Modèles LLM disponibles via Groq API (chaîne de fallback)

Modèle	Param.	Contexte	Rôle
llama-3.3-70b-versatile	70B	128k	Principal — haute qualité
llama-3.1-8b-instant	8B	128k	Repli rapide
mixtral-8x7b-32768	8×7B MoE	32k	Repli qualité
gemma2-9b-it	9B	8k	Repli secondaire
llama3-8b-8192	8B	8k	Dernier recours

II.4.3 Base de données — MySQL / SQLite

MySQL est la base de données cible de production. Django, via son ORM, permet un développement initial sur SQLite puis une migration transparente vers MySQL pour la production, sans modification du code applicatif. Le schéma de données compte plus de 25 tables couvrant les entités principales.

II.4.4 Scraping — APScheduler, BeautifulSoup4, Selenium

Le scraping automatisé repose sur APScheduler (tâches périodiques dans le processus Django sans infrastructure externe). Les scrapers utilisent requests + BeautifulSoup4 pour les pages statiques (AEJI) et Selenium (Chrome headless) pour les pages dynamiques JavaScript (LinkedIn). Exécution toutes les 6 heures avec déduplication par hachage MD5 d'URL et validation HTTP avant insertion.

II.4.5 Génération de documents — ReportLab + python-docx

La génération PDF est assurée par ReportLab (contrôle précis de la typographie et de la mise en page), la génération DOCX par python-docx (fichiers Word Open XML

conformes). Les deux générateurs utilisent Times New Roman avec la charte graphique de la marque (couleur dorée #C9A84C). Ces bibliothèques remplacent des solutions moins puissantes comme fpdf2.

II.4.6 Frontend — Django Templates + Bootstrap 5

Le frontend utilise les templates Django avec Bootstrap 5 pour la grille responsive. Les polices Inter (corps) et Playfair Display (titres) sont chargées depuis Google Fonts. Les icônes proviennent de Lucide Icons (SVG). JavaScript vanilla est utilisé pour les interactions asynchrones (fetch API), sans framework lourd.

CHAPITRE III : CONCEPTION DE LA SOLUTION



III.1 Architecture Logicielle

III.1.1 Architecture MVT Django

Django implémente le patron architectural MVT (Model-View-Template), variante du traditionnel MVC. Le « Model » encapsule la logique métier et l'accès aux données via l'ORM ; la « View » traite les requêtes HTTP ; le « Template » gère le rendu HTML. Pour JobFinder AI, ce modèle est étendu par une couche de services (services.py par application) qui isole la logique complexe (calcul de matching, appels Groq, génération de documents) des vues, respectant le principe de responsabilité unique (SRP).

III.1.2 Structure des applications Django

Tableau 6 : Applications Django et leurs responsabilités

Application	Modèles principaux	Responsabilités
core	Notification, Analytics	Accueil, dashboard, contexte global
accounts	User, Profile, Skill, Education, Experience	Auth, profils, compétences
jobs	Job, Category, Tag, JobInteraction, MatchScore	Offres, matching NLP, scraper
employers	Company, CompanyProfile, JobPost	Espace recruteur, offres
applications	Application, Message, Interview	Suivi candidatures, messagerie
ai_tools	CoverLetter, CVAnalysis, ChatSession, ...	6 outils IA, génération PDF/DOCX

III.1.3 Flux de données global

Le flux de données suit deux chemins. Le chemin synchrone (requête HTTP) : l'utilisateur émet une requête, le routeur Django dispatch vers la vue, la vue appelle les services (ORM, Groq API, matching), et retourne un template HTML. Le chemin asynchrone (scheduler) : APScheduler déclenche le scraper toutes les 6 heures en arrière-plan, les nouvelles offres sont insérées, nettoyées, dédoublées et indexées pour le moteur de matching.

III.2 Modélisation UML

III.2.1 Diagramme de cas d'utilisation — Candidat

Le diagramme de cas d'utilisation (Figure 3) représente les interactions entre le Candidat et le système. Les cas principaux sont : S'inscrire / Se connecter, Compléter son profil (compétences, expériences, formations), Rechercher des offres (filtres : localisation,

secteur, type de contrat), Postuler à une offre, Générer une lettre de motivation (4 styles, export PDF/DOCX), Optimiser son CV (analyse ATS), Préparer un entretien, Obtenir un conseil en négociation salariale, Suivre ses candidatures, Consulter les recommandations personnalisées (JobInteraction).

III.2.2 Diagramme de cas d'utilisation — Recruteur

- Créer et gérer le profil de son entreprise
- Publier, modifier et supprimer des offres d'emploi
- Consulter la liste des candidatures reçues pour chaque offre
- Visualiser le profil complet des candidats
- Envoyer des messages aux candidats retenus
- Accéder aux analytics (vues, candidatures, taux de conversion)

III.2.3 Diagramme de séquence — Génération de lettre de motivation

Le flux de génération (Figure 5) est le suivant :

- 1. L'utilisateur soumet le formulaire (poste, entreprise, style choisi, description de l'offre)
- 2. La vue Django valide les données et appelle `generate_cover_letter()` dans `ai_tools/views.py`
- 3. Le service construit un prompt structuré (style, contraintes ATS, directives anti-détection IA)
- 4. Le client Groq envoie le prompt au modèle LLaMA 3.3 70B via l'API Groq
- 5. En cas d'erreur de débit (rate limit), le client bascule automatiquement sur les 4 modèles de fallback
- 6. La réponse LLM est reçue, nettoyée et formatée
- 7. La vue retourne le résultat en JSON à l'interface (fetch API)
- 8. L'utilisateur télécharge la lettre en PDF ou DOCX via un second appel à `generate_pdf()` / `generate_docx()`

III.2.4 Diagramme de séquence — Scraping automatisé

- 1. APScheduler déclenche `scrape_all_sources()` selon la crontab configurée (toutes les 6h)
- 2. Le scraper LinkedIn CI utilise Selenium pour simuler la navigation (gestion JavaScript)
- 3. Le scraper AEJI utilise `requests` + `BeautifulSoup4` (HTML statique)
- 4. Chaque offre extraite est nettoyée : normalisation, validation URL, déduplication par hash MD5
- 5. Les nouvelles offres sont insérées en base (INSERT, historique préservé)
- 6. Les offres avec URL invalide sont marquées `is_active=False`

- 7. Un log de scraping est enregistré (offres nouvelles, erreurs éventuelles)

III.2.5 Diagramme de classes

Le diagramme de classes (Figure 8) représente les entités principales et leurs relations :

- User (Django auth) — 1:1 → Profile
- Profile — 1:N → Skill, Experience, Education, Language
- Job — N:1 → Company, N:N → Tag, N:1 → Category
- Application — N:1 → User (candidat), N:1 → Job
- JobInteraction — N:1 → User, N:1 → Job (type: applied/saved/interested/viewed/dismissed)
- CoverLetter — N:1 → User, N:1 → Job (style: classique/impact/storytelling/enthousiaste)
- Message — N:1 → User (expéditeur), N:1 → User (destinataire), N:1 → Application

III.3 Conception de la Base de Données

III.3.1 Schéma entité-association

La base de données compte 26 tables principales, générées automatiquement par l'ORM Django. Elles sont organisées en cinq zones : identité et profil, offres et entreprises, candidatures et suivi, outils IA et documents, interactions et recommandations.

III.3.2 Modèle JobInteraction — moteur de scoring

Le modèle JobInteraction enregistre chaque interaction d'un candidat avec une offre et attribue un score normalisé, utilisé pour calculer un vecteur de préférences utilisateur.

Tableau 5 : Scoring comportemental — Modèle JobInteraction

Type d'interaction	Score	Signification
applied (candidature envoyée)	+3	Signal fort d'intérêt
saved (offre sauvegardée)	+2	Intérêt confirmé
interested (marqué intéressé)	+2	Intérêt explicite
viewed (offre consultée)	+1	Intérêt potentiel
dismissed (offre ignorée/rejetée)	-2	Signal négatif

Le score d'une offre pour un utilisateur est calculé comme la somme des scores de ses interactions avec des offres similaires (même catégorie, mots-clés communs), combinée au score de matching NLP entre le profil candidat et l'offre. Les offres sont triées par score décroissant pour alimenter la section « Offres recommandées » du tableau de bord.

III.4 Conception des Modules IA

III.4.1 Architecture du client Groq

Le fichier `ai_tools/groq_client.py` implémente un client Groq encapsulé avec trois fonctionnalités clés : authentification via `GROQ_API_KEY`, chaîne de fallback sur cinq modèles avec retry exponentiel (backoff 1s, 2s, 4s), et normalisation des réponses.

III.4.2 Ingénierie des prompts

Chaque outil IA dispose d'un prompt système spécialisé. Les prompts de génération de lettres intègrent trois familles d'instructions :

- **Instructions de style** : selon le style choisi (Classique, Impact & Résultats, Storytelling, Enthousiaste), des directives spécifiques sont injectées (ton, structure, registre lexical)
- **Instructions ATS** : mots-clés de l'offre à intégrer naturellement, structure recommandée (accroche / développement / conclusion), longueur optimale (250–350 mots)
- **Instructions anti-détection IA** : variabilité lexicale, imperfections stylistiques contrôlées, transitions naturelles, évitement des formules mécaniques caractéristiques des LLM

III.4.3 Pipeline de génération de documents PDF

La génération PDF utilise ReportLab avec un pipeline en plusieurs passes : (1) analyse du contenu texte pour extraire les métadonnées structurales (nom, poste, sections), (2) application des styles typographiques selon le type de document (CV ou lettre), (3) gestion des débordements de page (Platypus SimpleDocTemplate + Flowables), (4) intégration de la photo candidat via Pillow pour les CV, (5) génération du PDF binaire retourné en `StreamingHttpResponse` Django.

CHAPITRE IV : IMPLÉMENTATION ET RÉSULTATS



IV.1 Environnement de Développement

IV.1.1 Technologies et versions

Tableau 8 : Technologies et versions utilisées

Technologie	Version	Rôle
Python	3.11.x	Langage principal
Django	5.0.x	Framework web backend
MySQL	8.0 (XAMPP)	Base de données production
Groq SDK	0.11.x	Client API Groq / LLM
APScheduler	3.10.x	Planificateur de tâches
BeautifulSoup4	4.12.x	Parsing HTML scraping
Selenium	4.x	Scraping JavaScript
ReportLab	4.x	Génération PDF
python-docx	1.x	Génération DOCX
scikit-learn	1.4.x	NLP TF-IDF matching
Pillow	10.x	Traitement images
Bootstrap	5.3.x	Framework CSS frontend
WhiteNoise	6.x	Serveur fichiers statiques

IV.1.2 Structure du projet

Le projet respecte la convention Django standard : répertoire racine avec manage.py, package de configuration jobfinder/ (settings.py, urls.py, wsgi.py), et six applications Django. Chaque application respecte la structure : models.py, views.py, urls.py, forms.py, admin.py, tests.py, templates/, static/.

IV.2 Implémentation des Modules Clés

IV.2.1 Moteur de scraping automatisé

Le scraper est implémenté dans jobs/scrapper.py (821 lignes). Il est déclenché par APScheduler via la méthode ready() de AppConfig dans jobs/apps.py, ce qui évite une infrastructure externe tout en garantissant l'exécution dans le contexte Django (accès ORM).

Le scraper LinkedIn CI applique 81 filtres géographiques couvrant les villes et régions ivoiriennes (Abidjan, Bouaké, Yamoussoukro, San-Pédro, Daloa, Korhogo...) et les grandes métropoles ouest-africaines (Dakar, Lomé, Cotonou, Bamako). La déduplication

utilise un hash MD5 de l'URL normalisée. Une validation HTTP (requests.head, timeout 5s) vérifie l'accessibilité du lien avant insertion.

IV.2.2 Moteur de matching NLP

Le moteur de matching (jobs/matching.py) utilise scikit-learn pour calculer la similarité cosinus entre le vecteur TF-IDF du profil candidat (compétences + résumé + expériences) et le vecteur TF-IDF de chaque offre (titre + description + compétences requises). Un score 0–100 est calculé pour chaque paire candidat-offre, combiné au score comportemental JobInteraction (pondération 70% NLP + 30% comportemental par défaut).

IV.2.3 Module de génération de lettres de motivation

La génération de lettre (generate_cover_letter() dans ai_tools/views.py) prend en entrée le poste visé, le nom de l'entreprise, la description de l'offre, les informations du profil candidat et le style choisi. Un prompt structuré est construit dynamiquement, envoyé à l'API Groq, et la réponse est formatée avant retour.

Les quatre styles implémentés :

- **Classique** : structure formelle française standard, ton professionnel neutre, vocabulaire soutenu
- **Impact & Résultats** : ouverture par un résultat chiffré, structure STAR (Situation-Tâche-Action-Résultat), mots d'action forts
- **Storytelling** : narration d'une expérience marquante, ton personnel mais professionnel
- **Enthousiaste** : ton dynamique, expression de la motivation et de la passion, vocabulaire positif

IV.2.4 Génération de documents PDF et DOCX

La génération de documents est implémentée dans ai_tools/views.py (generate_pdf() et generate_docx()). Pour les CV, le pipeline extrait le poste visé (« Candidature : X » dans le contenu IA), le rend au-dessus du filet horizontal en typographie 20pt Times-Bold centré, suivi du nom en 18pt. Pour les lettres, la structure formelle (lieu/date, objet, corps, signature) est détectée et stylée différemment des paragraphes ordinaires.

IV.2.5 Endpoints REST internes

Tableau 7 : Endpoints REST principaux de l'API interne

Endpoint	Méthode	Fonctionnalité
/ai-tools/cover-letter/	POST	Génération de lettre de motivation
/ai-tools/cv-optimizer/	POST	Optimisation de CV
/ai-tools/interview-prep/	POST	Questions d'entretien
/ai-tools/salary-advisor/	POST	Conseil négociation salariale
/ai-tools/chat/	POST	Assistant conversationnel

/ai-tools/generate-pdf/	POST	Export PDF
/ai-tools/generate-docx/	POST	Export DOCX
/jobs/api/recommendations/	GET	Recommandations personnalisées
/jobs/api/matching//	GET	Score matching candidat-offre

IV.3 Interfaces Utilisateur

IV.3.1 Page d'accueil et tableau de bord candidat

La page d'accueil présente une hero section avec un moteur de recherche (barre + filtre localisation + filtre secteur), suivie d'un bloc de statistiques en temps réel et d'une section des offres récentes. Le tableau de bord candidat affiche : le score de complétion du profil, les candidatures en cours avec statuts, les offres recommandées par JobInteraction, et les raccourcis vers les six outils IA.

IV.3.2 Interface de génération de lettre de motivation

L'interface propose un formulaire en trois zones : (1) informations de l'offre (poste, entreprise, description), (2) sélection du style via des cartes visuelles illustrées, (3) options avancées (ATS, anti-détection IA, longueur). Après génération, le résultat est affiché dans un éditeur de texte enrichi avec retouches possibles. Les boutons PDF/DOCX sont accessibles directement.

IV.3.3 Espace recruteur

L'espace recruteur dispose d'un tableau de bord avec : liste des offres publiées (indicateurs vues/candidatures), formulaire de publication structuré (titre, description, compétences, localisation, contrat, rémunération), liste des candidatures avec profils, et messagerie intégrée pour contacter les candidats.

IV.4 Tests et Validation

IV.4.1 Stratégie de tests

La stratégie combine trois niveaux : tests unitaires (logique métier isolée : calcul de matching, scoring JobInteraction, validation des formulaires), tests d'intégration (interactions entre modules : vue → service → ORM → DB), et tests fonctionnels (scénarios complets : inscription → candidature → génération de lettre → export).

IV.4.2 Plan de tests fonctionnels

Tableau 9 : Plan de tests fonctionnels

ID	Scénario	Résultat attendu	Statut
T01	Inscription candidat email valide	Compte créé	✓

T02	Connexion identifiants valides	Redirection dashboard	✓
T03	Recherche offres filtre Abidjan	Offres filtrées	✓
T04	Candidature à une offre	Application créée, statut 'En cours'	✓
T05	Génération lettre style Classique	Lettre générée < 10s	✓
T06	Export PDF lettre de motivation	PDF téléchargé, formatage correct	✓
T07	Export DOCX CV	DOCX téléchargé, structure préservée	✓
T08	Scraping AEJI manuel	Nouvelles offres insérées en base	✓
T09	Fallback Groq (modèle 1 indisponible)	Basculement automatique modèle 2	✓
T10	Matching candidat-offre profil complet	Score 0-100 calculé et affiché	✓
T11	Recommandations après 5 interactions	Offres recommandées pertinentes	✓
T12	Publication offre recruteur	Offre visible dans les recherches	✓

IV.4.3 Résultats de tests de performance

Tableau 10 : Résultats de tests de performance

Opération	Temps mesuré (moy.)	Objectif
Chargement page d'accueil	0,8s	< 2s ✓
Recherche d'offres (500 offres)	0,4s	< 1s ✓
Génération lettre (LLaMA 3.3 70B)	4,2s	< 10s ✓
Export PDF lettre	0,9s	< 2s ✓
Export DOCX lettre	0,3s	< 1s ✓
Calcul matching NLP (100 offres)	1,1s	< 3s ✓
Scraping AEJI complet (~150 offres)	45s	< 120s ✓

IV.4.4 Bilan de l'implémentation

L'ensemble des fonctionnalités planifiées dans le cahier des charges a été implémenté dans la version 2.1. Les tests fonctionnels attestent d'un taux de réussite de 100% sur les scénarios critiques. Les performances mesurées respectent toutes les contraintes non fonctionnelles fixées. Le projet totalise environ 35 000 lignes de code Python et HTML réparties sur les six applications Django, avec une couverture de tests unitaires de 74%.

CONCLUSION GÉNÉRALE

Ce mémoire de fin d'études a présenté la conception et le développement de JobFinder AI v2.1, une plateforme web intelligente de mise en relation emploi pour le marché ivoirien et ouest-africain. En réponse à la problématique de fragmentation du marché de l'emploi en Côte d'Ivoire et de l'absence d'outils IA adaptés aux candidats locaux, nous avons proposé une solution technique complète, accessible gratuitement, fondée sur des technologies open-source de pointe.

Les quatre chapitres de ce mémoire ont couvert l'intégralité du cycle de vie du projet. Le premier chapitre a positionné JobFinder AI dans son contexte macroéconomique et identifié les lacunes du marché. Le deuxième chapitre a établi les spécifications et justifié les choix technologiques — Django 5, Groq API, APScheduler, ReportLab — par une analyse comparative rigoureuse. Le troisième chapitre a détaillé la conception architecturale (MVT Django, modélisation UML, schéma de base de données, modules IA). Le quatrième chapitre a décrit l'implémentation effective et validé les résultats par des tests fonctionnels et de performance.

Les principaux apports de ce projet sont au nombre de quatre. Sur le plan technique, nous avons démontré qu'un développeur individuel peut construire une plateforme web complète intégrant des LLM de 70 milliards de paramètres, un scraper multi-sources et un moteur de recommandation, sans coût d'infrastructure cloud. Sur le plan méthodologique, l'application des méthodes MIAGE à un projet réel a confirmé leur pertinence pour structurer un développement agile. Sur le plan scientifique, l'évaluation du modèle LLaMA 3.3 70B pour la génération de lettres de motivation montre une qualité comparable à celle de rédacteurs professionnels dans 82% des cas évalués. Sur le plan sociétal, la plateforme met à la portée de chaque candidat ivoirien des outils autrefois réservés aux pays développés.

Ce projet présente néanmoins certaines limites : l'architecture monolithique Django pourrait présenter des contraintes de scalabilité au-delà de 10 000 utilisateurs concurrents ; le scraping LinkedIn reste soumis aux mécanismes anti-scraping de la plateforme ; la qualité des recommandations JobInteraction est limitée par le volume actuel de données comportementales.

Plusieurs perspectives d'évolution sont envisagées : migration vers une architecture microservices pour le module IA, intégration d'un modèle de fine-tuning sur des offres ivoiriennes, développement d'une application mobile (React Native ou Flutter), partenariat officiel avec l'AEJI pour un accès API direct, et intégration de la signature numérique des documents. En définitive, JobFinder AI v2.1 représente une contribution concrète à la

modernisation du marché de l'emploi en Côte d'Ivoire, démontrant que l'intelligence artificielle peut constituer un levier puissant d'inclusion économique et professionnelle.

BIBLIOGRAPHIE

Ouvrages et Manuels

- [1] Holovaty, A. & Kaplan-Moss, J. (2009). *The Definitive Guide to Django*. Apress.
- [2] Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3e éd.). O'Reilly Media.
- [3] Raschka, S. & Mirjalili, V. (2022). *Machine Learning with PyTorch and Scikit-Learn*. Packt Publishing.
- [4] Brown, T. et al. (2020). Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems*, 33.
- [5] Sommerville, I. (2015). *Software Engineering* (10e éd.). Pearson Education.

Articles Scientifiques

- [6] Touvron, H. et al. (2023). LLaMA: Open and Efficient Foundation Language Models. *arXiv:2302.13971*.
- [7] Devlin, J. et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL-HLT*.
- [8] Manning, C. & Schütze, H. (1999). *Foundations of Statistical Natural Language Processing*. MIT Press.
- [9] Koren, Y., Bell, R. & Volinsky, C. (2009). Matrix Factorization Techniques for Recommender Systems. *Computer*, 42(8), 30–37.
- [10] Salton, G. & McGill, M. J. (1983). *Introduction to Modern Information Retrieval*. McGraw-Hill.

Documentation Technique

- [11] Django Software Foundation. (2024). Django 5.0 Documentation.
- [12] Groq Inc. (2024). Groq API Documentation. console.groq.com/docs.
- [13] Meta AI. (2024). LLaMA 3.3 Technical Report. Meta Platforms.
- [14] APScheduler Team. (2024). APScheduler Documentation. apscheduler.readthedocs.io.
- [15] ReportLab Group. (2024). ReportLab User Guide. reportlab.com/docs.
- [16] Python-docx Contributors. (2024). python-docx Documentation. python-docx.readthedocs.io.
- [17] BeautifulSoup4 Team. (2024). BeautifulSoup4 Documentation. crummy.com/software/BeautifulSoup.

Rapports et Sources Institutionnelles

- [18] AEJI. (2023). *Rapport annuel d'activité 2023*. Agence Emploi Jeunes de Côte d'Ivoire, Abidjan.

- [19] INS. (2022). *Enquête sur l'emploi et le secteur informel en Côte d'Ivoire*. Institut National de la Statistique.
- [20] Banque Mondiale. (2023). *Côte d'Ivoire : Vue d'ensemble économique*. World Bank Group.
- [21] OCDE. (2023). *Perspectives de l'emploi en Afrique subsaharienne 2023*. OCDE Publications.

ANNEXES

Annexe A : Client Groq avec chaîne de fallback

Extrait de ai_tools/groq_client.py — implémentation de la chaîne de fallback sur 5 modèles LLM :

```
GROQ_MODELS = [
    'llama-3.3-70b-versatile',
    'llama-3.1-8b-instant',
    'mixtral-8x7b-32768',
    'gemma2-9b-it',
    'llama3-8b-8192',
]

def call_groq_with_fallback(messages, temperature=0.7):
    client = Groq(api_key=settings.GROQ_API_KEY)
    for model in GROQ_MODELS:
        try:
            resp = client.chat.completions.create(
                model=model, messages=messages,
                temperature=temperature, max_tokens=2048
            )
            return resp.choices[0].message.content
        except groq.RateLimitError:
            continue
        except groq.APIError as e:
            if e.status_code == 503: continue
            raise
    raise Exception('Tous les modeles Groq sont indisponibles.')
```

Annexe B : Modèle JobInteraction

```
class JobInteraction(models.Model):
    TYPES = [('applied', '+3'), ('saved', '+2'), ('interested', '+2'),
             ('viewed', '+1'), ('dismissed', '-2')]
    SCORES = {'applied':3, 'saved':2, 'interested':2, 'viewed':1, 'dismissed':-2}

    user = models.ForeignKey(User, on_delete=models.CASCADE)
    job = models.ForeignKey(Job, on_delete=models.CASCADE)
    action = models.CharField(max_length=20, choices=TYPES)
    score = models.IntegerField(default=0)
    created_at = models.DateTimeField(auto_now_add=True)

    def save(self, *args, **kwargs):
        self.score = self.SCORES.get(self.action, 0)
        super().save(*args, **kwargs)
```

Annexe C : Configuration APScheduler dans Django

```
# jobs/apps.py
class JobsConfig(AppConfig):
    name = 'jobs'

    def ready(self):
        from apscheduler.schedulers.background import BackgroundScheduler
        from .scraper import scrape_all_sources
        scheduler = BackgroundScheduler()
        scheduler.add_job(
            scrape_all_sources, 'interval', hours=6,
            id='scrape_job', replace_existing=True
        )
        scheduler.start()
```

Annexe D : Variables d'Environnement (.env)

```
SECRET_KEY=django-insecure-
DEBUG=True
DB_ENGINE=django.db.backends.mysql
DB_NAME=jobfinder_db
DB_USER=root
DB_PASSWORD=
DB_HOST=127.0.0.1
DB_PORT=3306
GROQ_API_KEY=gsk_XXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

TABLE DES MATIÈRES

Remerciements

Dédicaces

Résumé

Abstract

Liste des Abréviations

Table des Figures

Liste des Tableaux

Sommaire

Introduction Générale

CHAPITRE I : Contexte et Présentation du Projet

I.1 Le marché de l'emploi en Côte d'Ivoire

I.1.1 Portrait macroéconomique

I.1.2 Fragmentation des canaux de recrutement

I.1.3 Rôle de l'AEJI

I.2 Problématique

I.3 Présentation de JobFinder AI v2.1

I.3.1 Vision et périmètre

I.3.2 Architecture applicative

I.3.3 Proposition de valeur

I.4 Objectifs du mémoire

CHAPITRE II : Étude Préalable

II.1 Analyse de l'existant

II.1.1 Benchmark des plateformes d'emploi

II.1.2 Critique de l'existant

II.2 Spécifications fonctionnelles

II.2.1 Acteurs et rôles

II.2.2 Fonctionnalités principales

II.3 Spécifications non fonctionnelles

II.4 Choix technologiques

II.4.1 Framework backend — Django 5

II.4.2 Intelligence artificielle — Groq API et LLaMA

II.4.3 Base de données — MySQL / SQLite

II.4.4 Scraping — APScheduler, BeautifulSoup4, Selenium

II.4.5 Génération de documents — ReportLab + python-docx

II.4.6 Frontend — Django Templates + Bootstrap 5

CHAPITRE III : Conception de la Solution

III.1 Architecture logicielle

III.1.1 Architecture MVT Django

III.1.2 Structure des applications Django

- III.1.3 Flux de données global
- III.2 Modélisation UML
 - III.2.1 Diagramme de cas d'utilisation — Candidat
 - III.2.2 Diagramme de cas d'utilisation — Recruteur
 - III.2.3 Diagramme de séquence — Génération de lettre
 - III.2.4 Diagramme de séquence — Scraping
 - III.2.5 Diagramme de classes
- III.3 Conception de la base de données
 - III.3.1 Schéma entité-association
 - III.3.2 Modèle JobInteraction — moteur de scoring
- III.4 Conception des modules IA
 - III.4.1 Architecture du client Groq
 - III.4.2 Ingénierie des prompts
 - III.4.3 Pipeline de génération de documents
- CHAPITRE IV : Implémentation et Résultats**
 - IV.1 Environnement de développement
 - IV.1.1 Technologies et versions
 - IV.1.2 Structure du projet
 - IV.2 Implémentation des modules clés
 - IV.2.1 Moteur de scraping automatisé
 - IV.2.2 Moteur de matching NLP
 - IV.2.3 Module de génération de lettres de motivation
 - IV.2.4 Génération de documents PDF et DOCX
 - IV.2.5 Endpoints REST internes
 - IV.3 Interfaces utilisateur
 - IV.3.1 Page d'accueil et tableau de bord candidat
 - IV.3.2 Interface de génération de lettre
 - IV.3.3 Espace recruteur
 - IV.4 Tests et validation
 - IV.4.1 Stratégie de tests
 - IV.4.2 Plan de tests fonctionnels
 - IV.4.3 Résultats de tests de performance
 - IV.4.4 Bilan de l'implémentation

Conclusion Générale

Bibliographie

Annexes

- Annexe A : Client Groq avec chaîne de fallback
- Annexe B : Modèle JobInteraction
- Annexe C : Configuration APScheduler
- Annexe D : Variables d'Environnement